



TECHNISCHE  
UNIVERSITÄT  
WIEN

**ottobock.**

# Framework for System Simulation

**Thesis – Design and Implementation**

Florian Frech, 12.01.2026

## 1. Hybrid Co-Simulation: Motivation and Use-Case

- Controlled Pendulum with Wall Contact
- Problems with Windup during Wall Contact

## 2. Hybrid Co-Simulation: Theory

- Hybrid Model Formalism and Simulation Algorithm
- Step Revision
- Co-Simulation: it's about time

## 3. Framework Extensions

- Component Classification
- Event Indicator and Event Subscription
- CoSimComponent Interface for Hybrid Simulation
- System Level: Event Connection and Algorithm Selection
- History and Event Tracking

## 4. Event Handling Scenarios

- Non-simultaneous Events (Distinct Single Events)
- Weak Simultaneous Events (Event Chain)
- Strong Simultaneous Events (Multiple Events at same Superdense Time)

## 5. Hybrid Master Algorithm

- Global Hybrid Co-Simulation Step
- Event Detection
- Event Location
- Event Handling

## 6. FMI 2.0 – Hybrid Capabilities and Constraints

- FMI Rollback Functionality
- Solver Dependencies: Explicit Euler vs. CVode
- Workarounds for State Snapshots and State Restoring

## 7. Verification

- FMU Rollback and Zero-Crossing Detection
- Hybrid Controlled Pendulum with Wall Contact

## 8. Summary

- Core Capabilities
- Current Limitations

## 9. Next Steps

## Problem: Integrator Windup emerges for wall contact

- The pendulum follows a sinusoidal **reference trajectory** that **crosses beyond the wall position**
- When the reference moves past the wall, the **pendulum cannot follow** due to physical contact
- The tracking **error** (reference – actual position) continuously **accumulates** in the integrator
- The controller "wants" the pendulum to move through the wall, but this is physically impossible
- This accumulated error grows large during each wall contact period

## Integrator Windup

- Occurs when the integral term of a PID controller accumulates error during periods where the system cannot respond to the control commands
- Root Cause: Integrator continues summing error even when the actuator is saturated or the system is physically constrained

## Consequences of Integrator Windup

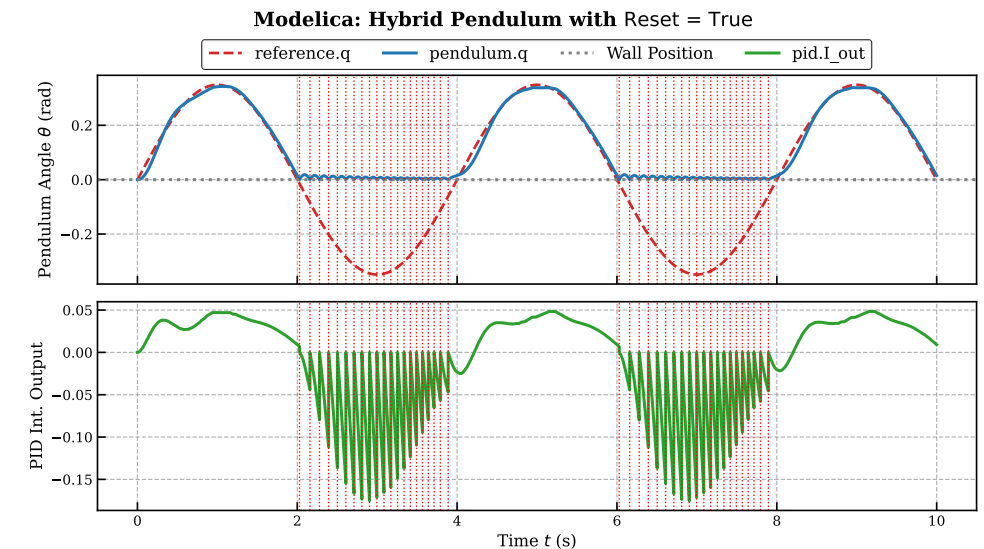
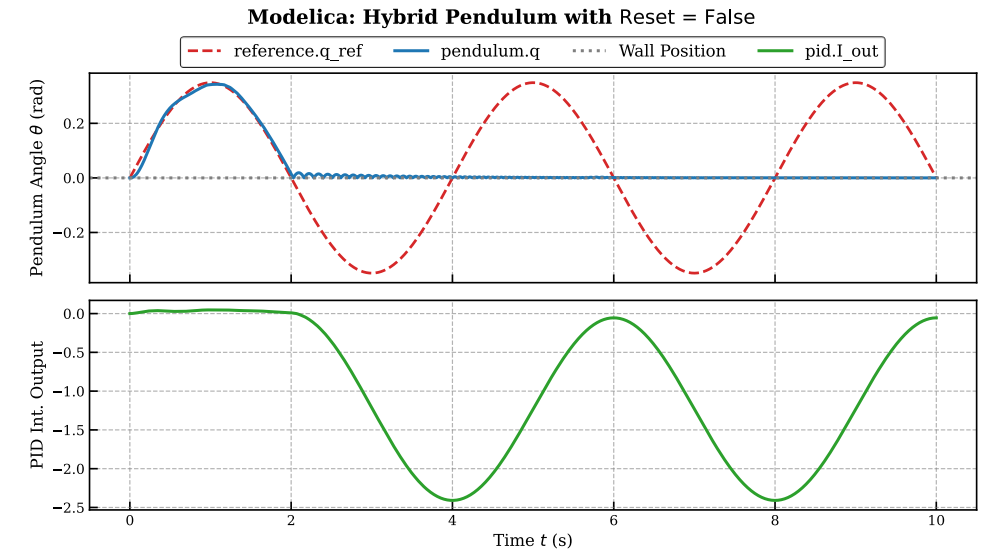
- When the reference trajectory returns to the feasible region, the **integrator has a large residual value**
- The pendulum fails to track the reference correctly after contact ends leads to a poor transient response

## Solution: Integrator Reset

- **Event-triggered reset:** The integrator is reset to zero on the rising edge of wall contact
- **Prevents error accumulation:** No integration occurs during physically constrained motion
- **Clean restart:** When contact ends, the integrator starts fresh with zero offset
- **Immediate tracking recovery:** The controller can respond correctly to the reference trajectory without fighting accumulated historical error

## Insight for Hybrid Co-Simulation:

- **Hybrid system behavior:** continuous dynamics (PID control) require discrete intervention (integrator reset) triggered by a physical event (wall contact)
- Proper handling requires **event detection** and **state reset** capabilities in the simulation framework





## Models of Time

### 1. Order of Events

- Notion of time must provide a **clear ordering of events**.
- Each component in a system must be able to **differ between past, present, and future**.
- The state of a component at a present time contains all information about the past
- Component state changes as time advances to the future
- All **observers should see state changes in the same order**

### 2. Causality

- Semantic notion of time needs to respect causality
- **If an event A causes an event B, then A must occur before B in time**
- The order must be observable for all observer

### 3. Simultaneity

- Two events are **simultaneous, if all observers see them occurring at the same time**
- Models where one observer deems two events to be simultaneous and another does not need to be avoided

## Requirements for Rigorous Time Model (according to Broman et. al)

1. **Uniform Precision:** Time resolution is constant across all time instants and identical for all observers to ensure detection of simultaneous events
2. **Magnitude-Independent Resolution:** Choice of  $t_0$  does not affect precision
3. **Associative Addition:**  $(t_1 + t_2) + t_3 = t_1 + (t_2 + t_3)$

## Time Resolution = Precision of Time Representation

- Time resolution is the **smallest representable time difference** between two timestamps.
- Time points between  $t$  and  $t + \Delta t_r$  **are indistinguishable** and cannot be represented
- Example: Resolution of 1 ms means that two timestamps are considered equal if

$$|t_1 - t_2| < 0.001s$$

## Newtonian Time Model $t \in \mathbb{R}_{\geq 0}$

### ■ Non-uniform precision:

- IEEE double precision relative error  $\epsilon_{machine} \approx 10^{-16}$
- Absolute precision degrades with magnitude  $\Delta t \propto |t|$  (loss of precision for large  $t$ )

### ■ Non-Associativity: $(t_1 + t_2) + t_3 \neq t_1 + (t_2 + t_3)$ order affects the final time stamp

### ■ Fragile Simultaneity: two components with same period $T$ starting at $t_0 = 0$ might not have the same final time stamp after $n$ steps due to rounding (arbitrary order destroys determinism)

### ■ Violation of Transitivity of Event Simultaneity:

- Using floating point numbers and tolerance to define simultaneity
- If events A and B are simultaneous within tolerance, and event B and C are simultaneous within tolerance, it is **possible that A and C are not simultaneous within tolerance**

## Superdense Time

- Superdense time can be represented as a pair  $(t, n)$ , where  $t$  represents the model time at which some event occurs and  $n$  is the index of simultaneous discrete events
  - Simultaneous in the **weak sense**:  $t_1 = t_2$
  - Simultaneous in the **strong sense**:  $t_1 = t_2, n_1 = n_2$
- Time stamps can be ordered in lexicographical order:  
 $(t_1, n_1) < (t_2, n_2)$  if  $t_1 < t_2$  or  $(t_1 = t_2 \text{ and } n_1 < n_2)$
- Event occurs at a superdense time. Events triggered by events at same model time have increasing index  $n$
- **Problem:** Representation of the model time as real number in computers

## Proposal of Cremona et. al: Integer Time for Model time $t$

- Represent the model time using an integer  $i \in \mathbb{Z}_{\geq 0}$  with resolution  $r = 10^n$   
$$t = i \cdot r = i \cdot 10^n s$$

## Advantages:

- **Exact Arithmetic:** Integer operations (add, subtract, multiply by integer) are exact
- **Uniform Precision:** Resolution independent of magnitude
- **Associativity:**  $(i_1 + i_2) + i_3 = i_1 + (i_2 + i_3)$
- **Transitivity of Simultaneity:** if  $i_1 = i_2$  and  $i_2 = i_3$  then  $i_1 = i_3$

## Component Classification:

Proper orchestration of a hybrid system requires the classification of the involved components into

1. **Event Sources:** Components with **event\_indicators** (rollback required)
  - Detected via **has\_state\_event\_property**
  - Components are stored in **system.event\_sources**
2. **Event Listeners:** Components subscribed to events (rollback optional)
  - Detected via **has\_event\_subscriptions** property
  - Components are stored in **system.event\_listeners**
  - Components may be an event source and an event listener
3. **Continuous-Only:** No hybrid capabilities
  - Components are stored in **system.continuous\_only**

## Event Indicator:

- Zero-crossing detection function with configurable direction (rising/falling/any)
- Evaluated on components state to detect discrete events during continuous simulation
- **Attributes:**
  - Callable function that is defined for a CoSimComponent and returns a float
  - Direction flag: -1 (falling), 0 (any), 1 (rising)
- **Evaluate method:** Evaluate the assigned event indicator function to detect sign changes
- Components register as **event sources** via **add\_event\_indicator(...)**
- An event output port is automatically created for event source components when indicators are registered

## Events and Event Subscription:

- Events are defined by an event name, the name of the source component, and a direction flag
- Components register as **event listeners** via **subscribe\_event()**

## CoSimComponent Interface

- **snapshot\_state():** Returns current snapshot of component's internal state
- **restore\_state():** Pure rollback to previous state at time t
- **support\_rollback** property: Dynamic check if component implements rollback
- Only **event sources must support rollback**, listeners may be rollback-free
- **\_handle\_events\_internal():** Defines the method which is executed for a specific event type

## Event Connection System:

- **EventConnection** data class links event sources to listeners
- System maintains **\_events\_targets\_by\_source** mapping for event dispatch
- Validated that listeners are subscribed to events they are connected to

## Automatic Algorithm Selection

- System inspects **event\_sources** during initialization
- If an event source exists, the system automatically selects **HybridAlgorithm**
- As default, if the system does not contain any hybrid components, it uses the **GaussSeidelAlgorithm**

## History and Event Tracking

- **Component History Extension**
  - Event ports are automatically added to history tracking when an indicator is successfully registered
  - Event occurrence timestamps are stored separately from continuous port data
  - Retrieval via **get\_history()** includes event timings
- **System-Level Event History**
  - **record\_event(comp\_name, event\_name, t)** for centralized event logging
  - **get\_all\_event\_histories()** returns {(comp, event):[t1, t2, ...]} dictionary
  - Integrated with component histories for complete simulation timeline

## System Overview:

- Demonstrates a hybrid system where **two events occur at distinct times** (due to chosen parameters)
- This example verifies that the algorithm correctly detects and handles each event sequentially

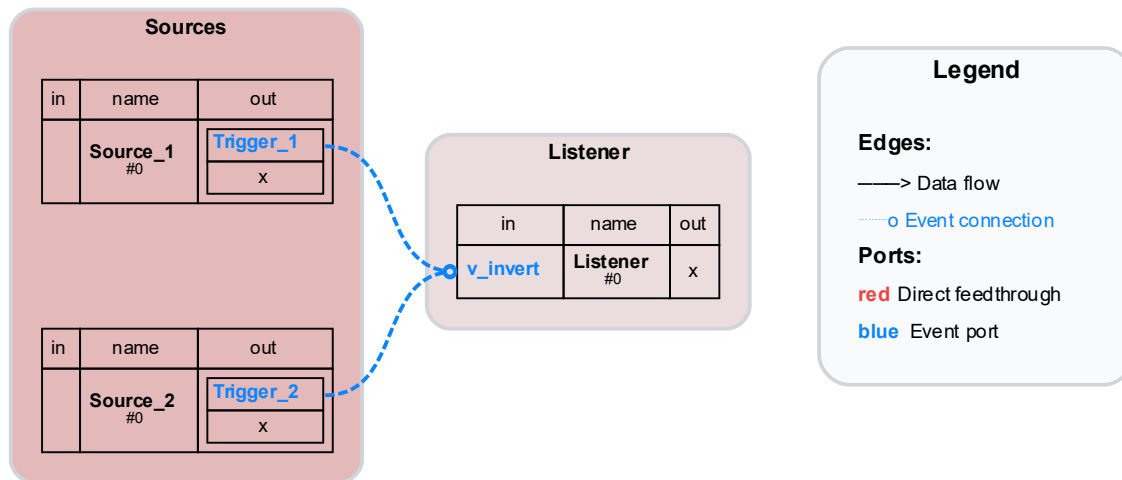
## Hybrid Source Component:

- Simple Component with linear dynamics:  $x(t) = x_0 + v \cdot (t - t_0)$
- **Rollback support:** Implements `snapshot_state()` and `restore_state()` required for event sources to perform bisection search for the event time
- **Event indicator:** Returns the current position `x` for zero-crossing detection

## Hybrid Listener:

- Component with identical linear dynamics that **respond to events by inverting its velocity**
- **No rollback required:** Event listeners do not need to restore state
- **Discontinuity:** Velocity inversion creates discontinuity in dynamics (kink in `x`, jump in `v`)

## Simple Hybrid Test System



```

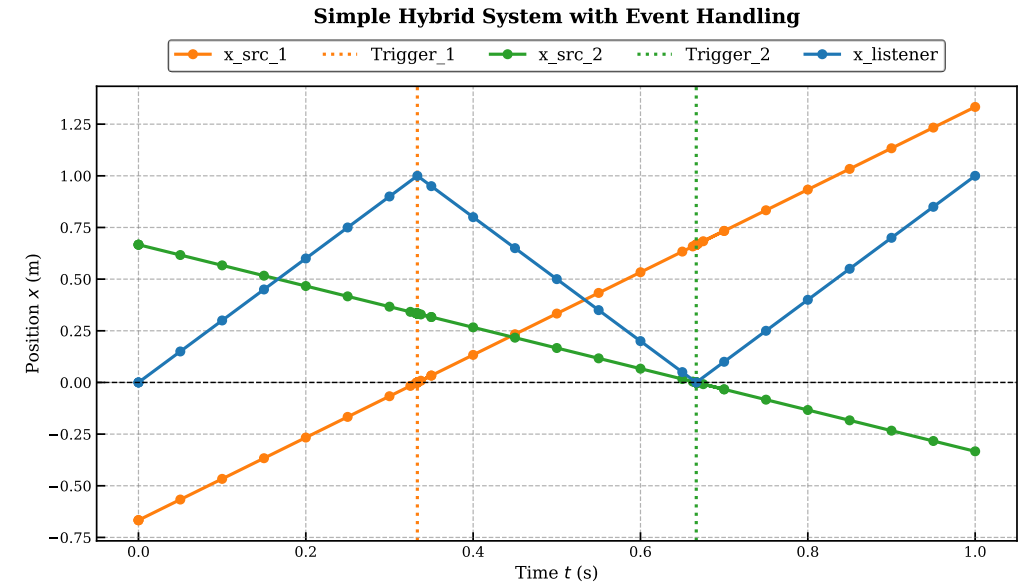
1 class HybridSource(CoSimComponent):
2     """
3     Simple component with a state variable that evolves linearly.
4     x(t) = x0 + v * (t - t0)
5     """
6     def __init__(self, name: str, x0: float = -1.0, v: float = 1.0, t0: float = 0.0):
7         super().__init__(name, group='Sources')
8         self.x0 = x0; self.v = v; self.t0 = t0
9
10    def _initialize_component(self, t0: float) -> None:
11        self.x = self.x0
12        self.output_specs.update({"x": PortSpec(name="x",
13                                                type=PortType.REAL,
14                                                direction="out")})
15
16    def _do_step_internal(self, t: float, dt: float) -> None:
17        self.x = self.x0 + self.v * (t + dt - self.t0)
18
19    def snapshot_state(self):
20        return {"t": self.t, "x": self.x, "v": self.v, "t0": self.t0}
21
22    def restore_state(self, snapshot) -> None:
23        self.t = snapshot["t"]
24        self.t0 = snapshot["t0"]
25        self.x = snapshot["x"]
26        self.v = snapshot["v"]
27
28 class HybridListener(CoSimComponent):
29     """
30     Simple component with a state variable that evolves linearly.
31     x(t) = x0 + v * (t - t0)
32     """
33     def __init__(self, name: str, x0: float = 0.0, v: float = 1.0, t0: float = 0.0):
34         super().__init__(name, group="Listener")
35         self.x0 = x0; self.v = v; self.t0 = t0
36
37     def _initialize_component(self, t0: float) -> None:
38        self.x = self.x0
39        self.output_specs.update({"x": PortSpec(name="x",
40                                                type=PortType.REAL,
41                                                direction="out")})
42        self.input_specs.update({"v_invert": PortSpec(name="v_invert",
43                                                    type=PortType.EVENT,
44                                                    direction="in")})
45
46    def _do_step_internal(self, t: float, dt: float) -> None:
47        self.x = self.x0 + self.v * (t + dt - self.t0)
48
49    def _handle_events_internal(self, event_names, t):
50        if 'Trigger_1' in event_names or 'Trigger_2' in event_names:
51            self.t0 = t
52            self.x0 = self.x
53            self.v = -self.v

```

```

1 # 1) Create components
2 src_comp_1 = HybridSource(name="Source_1", x0=-2/3, v=2)
3 src_comp_2 = HybridSource(name="Source_2", x0=2/3, v=-1)
4 listener_comp = HybridListener(name="Listener", x0=0.0, v=3.0)
5
6 # 2) Define event indicator functions
7 def event_indicator_function(comp: HybridSource) -> float:
8     return comp.x
9
10 # 3) Create events
11 event_1 = Event(name="Trigger_1", source=src_comp_1.name, direction=1)
12 event_2 = Event(name="Trigger_2", source=src_comp_2.name, direction=-1)
13
14 # 4) Initialize event source components
15 src_comp_1.initialize(t0=0.0)
16 src_comp_2.initialize(t0=0.0)
17
18 # 5) Add event indicators to source components
19 src_comp_1.add_event_indicator(name=event_1.name,
20                               func=event_indicator_function,
21                               direction=event_1.direction)
22 src_comp_2.add_event_indicator(name=event_2.name,
23                               func=event_indicator_function,
24                               direction=event_2.direction)
25
26 # 6) Initialize listener component and subscribe to events
27 listener_comp.initialize(t0=0.0)
28 listener_comp.subscribe_event(event=event_1)
29 listener_comp.subscribe_event(event=event_2)
30
31 # 7) Create system
32 system = System(name="Hybrid System - Event Handling")
33
34 # 8) Add components to system
35 system.add_component(src_comp_1)
36 system.add_component(src_comp_2)
37 system.add_component(listener_comp)
38
39 # 9) Create and add event connections to the system
40 connection_1 = EventConnection(src_comp=src_comp_1.name,
41                               src_port=src_comp_1.output_specs['Trigger_1'].name,
42                               dst_comp=listener_comp.name,
43                               dst_port=listener_comp.input_specs['v_invert'].name)
44 connection_2 = EventConnection(src_comp=src_comp_2.name,
45                               src_port=src_comp_2.output_specs['Trigger_2'].name,
46                               dst_comp=listener_comp.name,
47                               dst_port=listener_comp.input_specs['v_invert'].name)
48 system.add_event_connection(connection_1)
49 system.add_event_connection(connection_2)
50
51 # 10) Initialize and run the system
52 system.initialize(t0=0.0)
53 system.run(t0=0.0, tf=1.0, dt = 0.05)

```



Component Classification:

- event\_sources: ['Source\_1', 'Source\_2']
- event\_listeners: ['Listener']
- continuous\_only: []

Using Algorithm: Hybrid-Algorithm

```

=====
Event(s) detected in interval [0.30000000, 0.35000000]
Located at t=0.33333334
Events: [('Source_1', 'Trigger_1')]
Dispatching event 'Trigger_1' from 'Source_1' to targets: ['Listener']
Listener handled event at t=0.3333: v inverted from 3.0 to -3.0
=====

```

```

=====
Event(s) detected in interval [0.65000000, 0.70000000]
Located at t=0.66666667
Events: [('Source_2', 'Trigger_2')]
Dispatching event 'Trigger_2' from 'Source_2' to targets: ['Listener']
Listener handled event at t=0.6667: v inverted from -3.0 to 3.0
=====

```

## System Overview:

- Demonstrates a hybrid system where a first event triggers a second event at the same real time instant
- System introduces the concept of superdense time where events are handled iteratively at same real time instant but with increasing micro step index (t, 0), (t, 1), ...
- This example verifies that the algorithm correctly detects and handles chained events

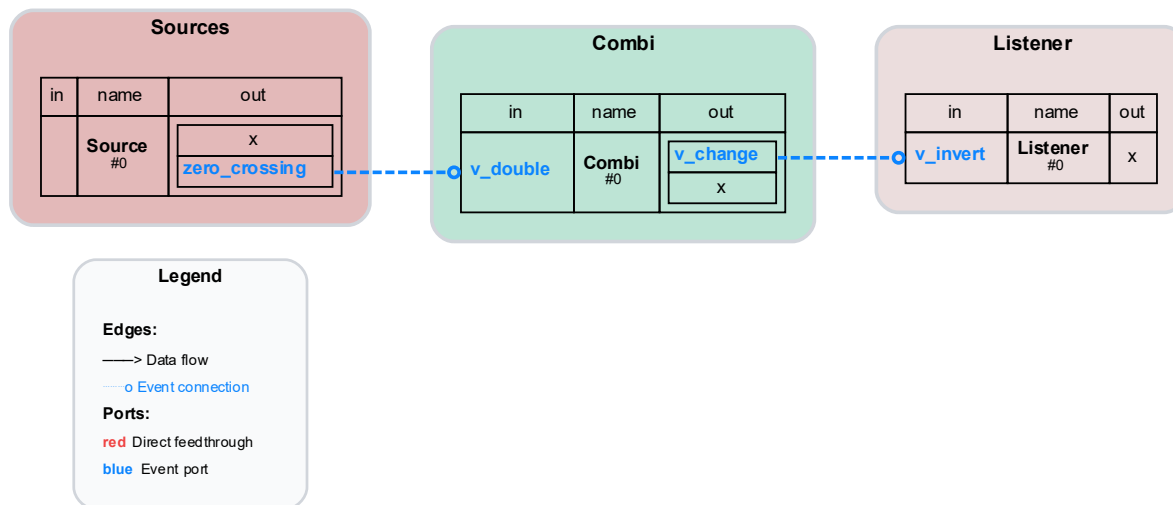
## Hybrid Combi Component:

- Simple Component with linear dynamics:  $x(t) = x_0 + v_{curr} \cdot (t - t_0)$
- Acts as both an **event listener** and **event source** (supports rollback)
- **Event Handling:** receives zero-crossing event and doubles its velocity
- **Event Indicator:** If the velocity changes, the  $v\_change$  event is emitted

## Hybrid Listener:

- Component with identical linear dynamics that **respond to  $v\_change$  event by inverting its velocity**

## Hybrid System - Event Chain



```

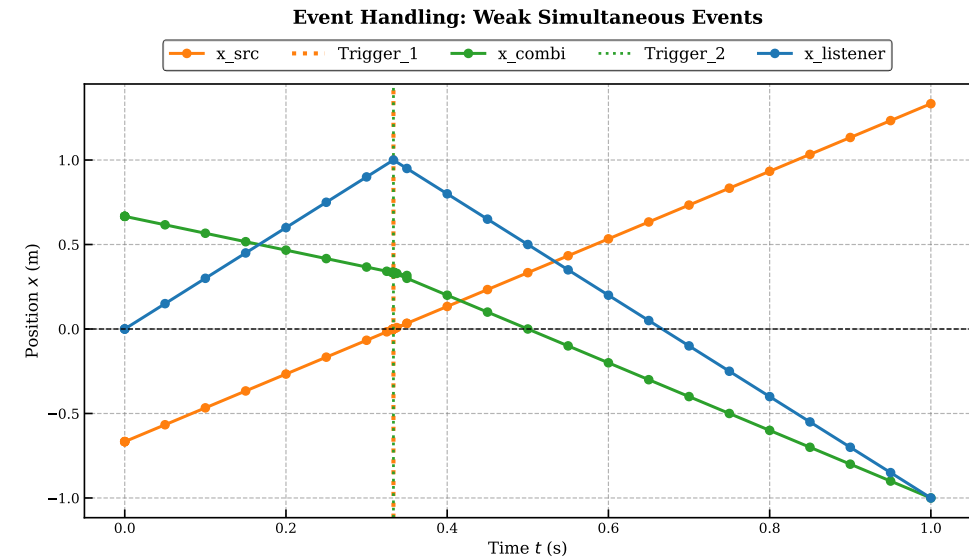
1 class HybridCombi(CoSimComponent):
2     def __init__(self, name: str, x0: float = 0.0, v: float = 1.0, t0: float = 0.0):
3         super().__init__(name, group='Combi')
4         self.x0 = x0
5         self.v_prev = v
6         self.v_curr = v
7         self.t0 = t0
8
9     def initialize_component(self, t0: float) -> None:
10        self.x = self.x0
11        self.input_specs.update({"v_double": PortSpec(name="v_double", type=PortType.EVENT, direction="in")})
12        self.output_specs.update({"x": PortSpec(name="x", type=PortType.REAL, direction="out")})
13        self._initialize_ports_from_specs()
14
15    def _do_step_internal(self, t: float, dt: float) -> None:
16        self.x = self.x0 + self.v_curr * (t + dt - self.t0)
17
18    def snapshot_state(self):
19        return {"t": self.t, "x": self.x, "v": self.v_curr, "v_prev": self.v_prev, "t0": self.t0}
20
21    def _handle_events_internal(self, event_names, t):
22        if 'zero_crossing' in event_names:
23            self.t0 = t
24            self.x0 = self.x
25            self.v_prev = self.v_curr
26            self.v_curr = 2 * self.v_curr
27            print(f"{self.name} handled event at t={t:.4f}: v doubled from {self.v_prev} to {self.v_curr}")
28
29    def restore_state(self, snapshot) -> None:
30        self.t = snapshot["t"]
31        self.t0 = snapshot["t0"]
32        self.x = snapshot["x"]
33        self.v_curr = snapshot["v"]
34        self.v_prev = snapshot.get("v_prev", self.v_curr)
35
36 class HybridListener(CoSimComponent):
37     def __init__(self, name: str, x0: float = 0.0, v: float = 1.0, t0: float = 0.0):
38         super().__init__(name, group="Listener")
39         self.x0 = x0; self.v = v; self.t0 = t0
40
41    def initialize_component(self, t0: float) -> None:
42        self.x = self.x0
43        self.output_specs.update({"x": PortSpec(name="x", type=PortType.REAL, direction="out")})
44        self.input_specs.update({"v_invert": PortSpec(name="v_invert", type=PortType.EVENT, direction="in")})
45        self._initialize_ports_from_specs()
46
47    def _do_step_internal(self, t: float, dt: float) -> None:
48        self.x = self.x0 + self.v * (t + dt - self.t0)
49
50    def update_output_states(self, t: Optional[float]=None) -> None:
51        self.outputs["x"].set(self.x, t=t)
52
53    def _handle_events_internal(self, event_names, t):
54        if 'v_change' in event_names:
55            self.t0 = t
56            self.x0 = self.x
57            self.v = -self.v
58            print(f"{self.name} handled event at t={t:.4f}: v inverted from { -self.v } to { self.v }")

```

```

1 # 1) Create components
2 src_comp = HybridSource(name="Source", x0=-2/3, v=2)
3 combi_comp = HybridCombi(name="Combi", x0=2/3, v=-1)
4 listener_comp = HybridListener(name="Listener", x0=0.0, v=3.0)
5
6 # 2) Define event indicator functions
7 def event_indicator_zero_crossing(comp: HybridSource) -> float:
8     return comp.x
9 def event_indicator_v_change(comp: HybridCombi) -> float:
10    return 0.0 if comp.v_curr != comp.v_prev else 1.0
11
12 # 3) Create events
13 event_1 = Event(name="zero_crossing", source=src_comp.name, direction=1)
14 event_2 = Event(name="v_change", source=combi_comp.name, direction=0)
15
16 # 4) Initialize event source components
17 src_comp.initialize(t0=0.0)
18 combi_comp.initialize(t0=0.0)
19
20 # 5) Add event indicators to source components
21 src_comp.add_event_indicator(name=event_1.name,
22                             func=event_indicator_zero_crossing,
23                             direction=event_1.direction)
24 combi_comp.add_event_indicator(name=event_2.name,
25                               func=event_indicator_v_change,
26                               direction=event_2.direction)
27
28 # 6) Initialize listener component and subscribe to events
29 combi_comp.subscribe_event(event=event_1)
30 listener_comp.initialize(t0=0.0)
31 listener_comp.subscribe_event(event=event_2)
32
33 # 7) Create system
34 system = System(name="Hybrid System - Event Chain")
35
36 # 8) Add components to system
37 system.add_component(src_comp)
38 system.add_component(combi_comp)
39 system.add_component(listener_comp)
40
41 # 9) Create and add event connections to the system
42 connection_1 = EventConnection(src_comp=src_comp.name,
43                                src_port=src_comp.output_specs['zero_crossing'].name,
44                                dst_comp=combi_comp.name,
45                                dst_port=combi_comp.input_specs['v_double'].name)
46 connection_2 = EventConnection(src_comp=combi_comp.name,
47                                src_port=combi_comp.output_specs['v_change'].name,
48                                dst_comp=listener_comp.name,
49                                dst_port=listener_comp.input_specs['v_invert'].name)
50
51 system.add_event_connection(connection_1)
52 system.add_event_connection(connection_2)
53 # 10) Initialize and run the system
54 system.initialize(t0=0.0)
55 system.run(t0=0.0, tf=1.0, dt = 0.05)

```



```

Component Classification:
- event_sources: ['Source', 'Combi']
- event_listeners: ['Combi', 'Listener']
- continuous_only: []

Using Algorithm: Hybrid-Algorithm

=====
Events detected in interval [0.30000000, 0.35000000]
Events: [('Source', 'zero_crossing')]
Located at t=(0.33333334, 0)

Handling events at (0.33333334, 0): [('Source', 'zero_crossing')]

Events grouped by component for handling: {'Combi': ['zero_crossing'], 'Listener': []}

Dispatching event 'zero_crossing' from 'Source' to targets: ['Combi']
Combi handled event at t=0.3333: v doubled from -1 to -2

New events detected after handling: [('Combi', 'v_change')]

Handling events at (0.33333334, 1): [('Combi', 'v_change')]

Events grouped by component for handling: {'Combi': [], 'Listener': ['v_change']}

Dispatching event 'v_change' from 'Combi' to targets: ['Listener']
=====

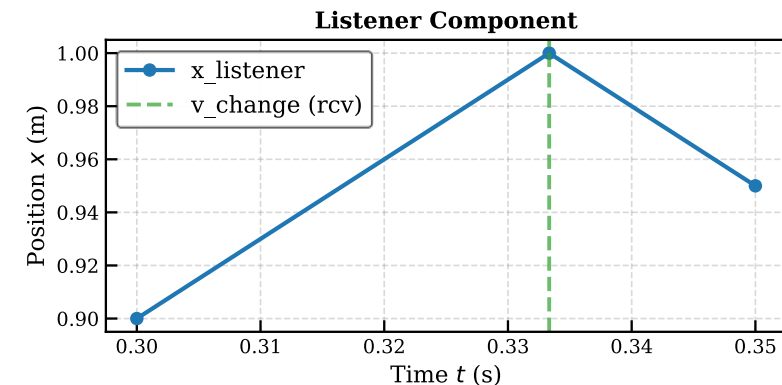
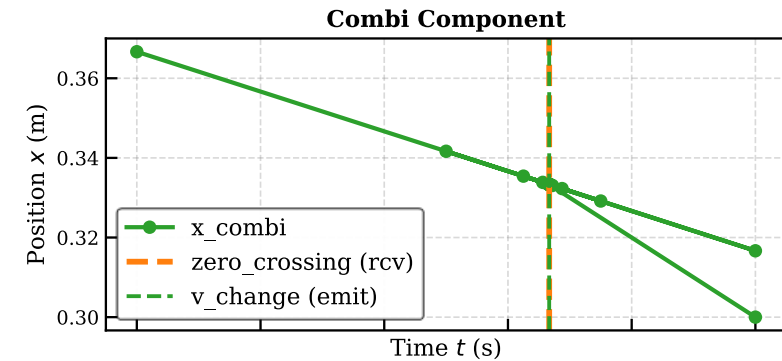
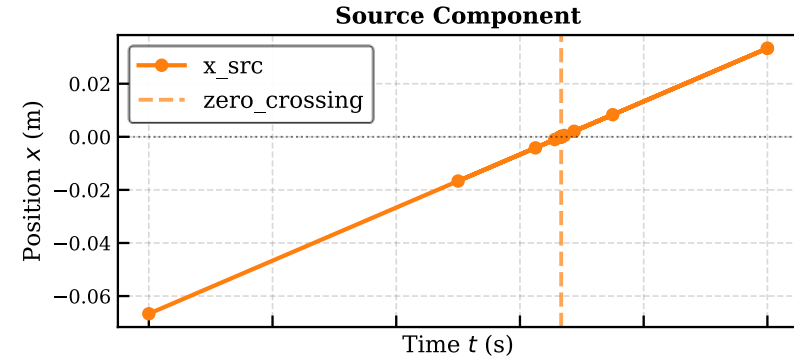
```

```

1 # 1) Create components
2 src_comp = HybridSource(name="Source", x0=-2/3, v=2)
3 combi_comp = HybridCombi(name="Combi", x0=2/3, v=-1)
4 listener_comp = HybridListener(name="Listener", x0=0.0, v=3.0)
5
6 # 2) Define event indicator functions
7 def event_indicator_zero_crossing(comp: HybridSource) -> float:
8     return comp.x
9 def event_indicator_v_change(comp: HybridCombi) -> float:
10    return 0.0 if comp.v_curr != comp.v_prev else 1.0
11
12 # 3) Create events
13 event_1 = Event(name="zero_crossing", source=src_comp.name, direction=1)
14 event_2 = Event(name="v_change", source=combi_comp.name, direction=0)
15
16 # 4) Initialize event source components
17 src_comp.initialize(t0=0.0)
18 combi_comp.initialize(t0=0.0)
19
20 # 5) Add event indicators to source components
21 src_comp.add_event_indicator(name=event_1.name,
22                             func=event_indicator_zero_crossing,
23                             direction=event_1.direction)
24 combi_comp.add_event_indicator(name=event_2.name,
25                               func=event_indicator_v_change,
26                               direction=event_2.direction)
27
28 # 6) Initialize listener component and subscribe to events
29 combi_comp.subscribe_event(event=event_1)
30 listener_comp.initialize(t0=0.0)
31 listener_comp.subscribe_event(event=event_2)
32
33 # 7) Create system
34 system = System(name="Hybrid System - Event Chain")
35
36 # 8) Add components to system
37 system.add_component(src_comp)
38 system.add_component(combi_comp)
39 system.add_component(listener_comp)
40
41 # 9) Create and add event connections to the system
42 connection_1 = EventConnection(src_comp=src_comp.name,
43                               src_port=src_comp.output_specs['zero_crossing'].name,
44                               dst_comp=combi_comp.name,
45                               dst_port=combi_comp.input_specs['v_double'].name)
46 connection_2 = EventConnection(src_comp=combi_comp.name,
47                               src_port=combi_comp.output_specs['v_change'].name,
48                               dst_comp=listener_comp.name,
49                               dst_port=listener_comp.input_specs['v_invert'].name)
50
51 system.add_event_connection(connection_1)
52 system.add_event_connection(connection_2)
53 # 10) Initialize and run the system
54 system.initialize(t0=0.0)
55 system.run(t0=0.0, tf=1.0, dt = 0.05)

```

Event Handling - Weak Simultaneous Events



## System Overview:

- Demonstrates a hybrid system where a two events occur at exactly the same real time instant (strong simultaneity)
- Both events modify the same state variable in a single listener component
- This example verifies that the algorithm correctly handles such conflicts by **checking commutativity of event handlers** before execution

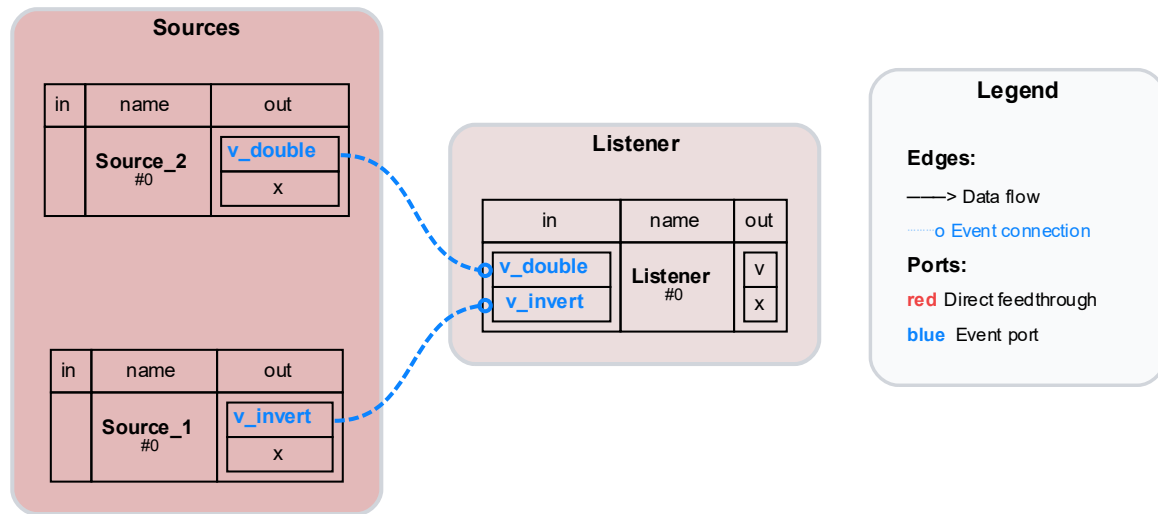
## Hybrid Source Component:

- Simple Component with same linear dynamics:  $x(t) = x_0 + v \cdot (t - t_0)$
- **Event Indicators:** Return position  $x$  for zero-crossing detection

## Event Listener Component:

- Can receive multiple simultaneous events affecting the same state variable
- **$v\_invert$**  event inverts the velocity
- **$v\_double$**  event doubles the velocity
- **Event handler commute:**  $(2v) = 2(-v) = -2v$

## Hybrid System - Event Commutativity



```

1 class HybridListener(CoSimComponent):
2     def __init__(self, name: str, x0: float = 0.0, v: float = 1.0, t0: float = 0.0, use_event_annotations: bool = False):
3         super().__init__(name, group="Listener")
4         self.x0 = x0; self.v = v; self.t0 = t0
5
6         if use_event_annotations:
7             self.event_commutativity.update({'v_invert', 'v_double': True})
8
9     def initialize_component(self, t0: float) -> None:
10        self.x = self.x0
11        self.input_specs.update({'v_invert': PortSpec(name="v_invert", type=PortType.EVENT, direction="in"),
12                                'v_double': PortSpec(name="v_double", type=PortType.EVENT, direction="in")})
13        self.output_specs.update({'x': PortSpec(name="x", type=PortType.REAL, direction="out"),
14                                  'v': PortSpec(name="v", type=PortType.REAL, direction="out")})
15        self._initialize_ports_from_specs()
16
17    def _do_step_internal(self, t: float, dt: float) -> None:
18        self.x = self.x0 + self.v * (t + dt - self.t0)
19
20    def _update_output_states(self, t: Optional[float]=None, event_names: Optional[list]=None) -> None:
21        self.outputs["x"].set(self.x, t=t)
22        self.outputs["v"].set(self.v, t=t)
23
24    def snapshot_state(self):
25        state = {"t": self.t, "x": self.x, "v": self.v, "t0": self.t0}
26        return state
27
28    def restore_state(self, snapshot, t):
29        self.t = snapshot["t"]
30        self.t0 = snapshot["t0"]
31        self.x = snapshot["x"]
32        self.v = snapshot["v"]
33
34    def _handle_events_internal(self, event_names, t):
35        if 'v_invert' in event_names:
36            self.t0 = t
37            self.x0 = self.x
38            self.v = -self.v
39            print(f"{self.name} handled event at t={t:.4f}: v inverted from { -self.v } to { self.v }")
40        if 'v_double' in event_names:
41            self.t0 = t
42            self.x0 = self.x
43            self.v = 2 * self.v
44            print(f"{self.name} handled event at t={t:.4f}: v doubled to { self.v }")

```

## Checking commutativity via annotations:

- User can annotate which event handler act on which state variables
- User defines which events commute, and which don't

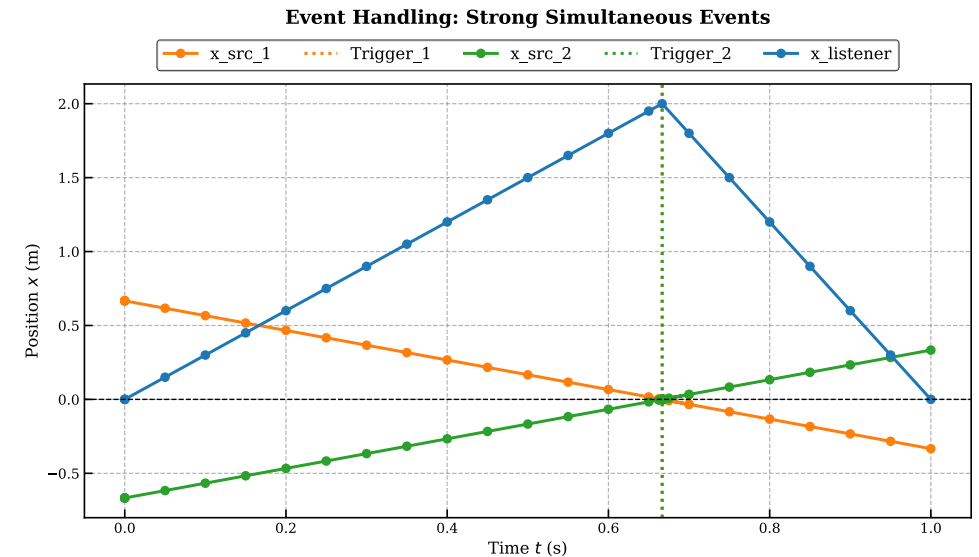
## Checking commutativity dynamically:

- All permutations (possible order of the simultaneous events) are applied to the listener
- It is checked if the final state is independent of the execution order (requires rollback)

```

1 # 1) Create components
2 src_comp_1 = HybridSource(name="Source_1", x0=2/3, v=-1)
3 src_comp_2 = HybridSource(name="Source_2", x0=-2/3, v=1)
4 listener_comp = HybridListener(name="Listener", x0=0.0, v=3.0)
5
6 # 2) Define event indicator functions
7 def event_indicator_zero_crossing(comp: HybridSource) -> float:
8     return comp.x
9
10 # 3) Create events
11 event_1 = Event(name="v_invert", source=src_comp_1.name, direction=-1)
12 event_2 = Event(name="v_double", source=src_comp_2.name, direction=1)
13
14 # 4) Initialize event source components
15 src_comp_1.initialize(t0=0.0)
16 src_comp_2.initialize(t0=0.0)
17
18 # 5) Add event indicators to source components
19 src_comp_1.add_event_indicator(name=event_1.name,
20                               func=event_indicator_zero_crossing,
21                               direction=event_1.direction)
22 src_comp_2.add_event_indicator(name=event_2.name,
23                               func=event_indicator_zero_crossing,
24                               direction=event_2.direction)
25
26 # 6) Initialize listener component and subscribe to events
27 listener_comp.initialize(t0=0.0)
28 listener_comp.subscribe_event(event=event_1)
29 listener_comp.subscribe_event(event=event_2)
30
31 # 7) Create system
32 system = System(name="Hybrid System - Event Commutativity")
33
34 # 8) Add components to system
35 system.add_component(src_comp_1)
36 system.add_component(src_comp_2)
37 system.add_component(listener_comp)
38
39 # 9) Create and add event connections to the system
40 connection_1 = EventConnection(src_comp=src_comp_1.name,
41                               src_port=src_comp_1.output_specs['v_invert'].name,
42                               dst_comp=listener_comp.name,
43                               dst_port=listener_comp.input_specs['v_invert'].name)
44 connection_2 = EventConnection(src_comp=src_comp_2.name,
45                               src_port=src_comp_2.output_specs['v_double'].name,
46                               dst_comp=listener_comp.name,
47                               dst_port=listener_comp.input_specs['v_double'].name)
48 system.add_event_connection(connection_1)
49 system.add_event_connection(connection_2)
50
51 # 10) Initialize the system
52 system.initialize(t0=0.0)
53 system.run(t0=0.0, tf=1.0, dt = 0.05)

```



```

Component Classification:
- event_sources: ['Source_1', 'Source_2']
- event_listeners: ['Listener']
- continuous_only: []

Using Algorithm: Hybrid-Algorithm

Starting hybrid co-simulation...

=====
Events detected in interval [0.65000000, 0.70000000]
Events: [('Source_1', 'v_invert'), ('Source_2', 'v_double')]
Located at t=(0.66666667, 0)

Handling events at (0.66666667, 0): [('Source_1', 'v_invert'), ('Source_2', 'v_double')]

Events grouped by component for handling: {'Listener': ['v_invert', 'v_double']}

Checking commutativity for events ['v_invert', 'v_double'] on component Listener...
Verifying dynamically ...
Listener handled 'v_invert' event at t=0.6667: v_prev = 3.0, v_curr = -3.0
Listener handled 'v_double' event at t=0.6667: v_prev = -3.0, v_curr = -6.0
Listener handled 'v_double' event at t=0.6667: v_prev = 3.0, v_curr = 6.0
Listener handled 'v_invert' event at t=0.6667: v_prev = 6.0, v_curr = -6.0
Event handlers ['v_invert', 'v_double'] on component Listener verified as commutative via dynamic check.

Dispatching event 'v_invert' from 'Source_1' to targets: ['Listener']
Listener handled 'v_invert' event at t=0.6667: v_prev = 3.0, v_curr = -3.0

Dispatching event 'v_double' from 'Source_2' to targets: ['Listener']
Listener handled 'v_double' event at t=0.6667: v_prev = -3.0, v_curr = -6.0
=====

```

## Global Stepping Logic and Procedure

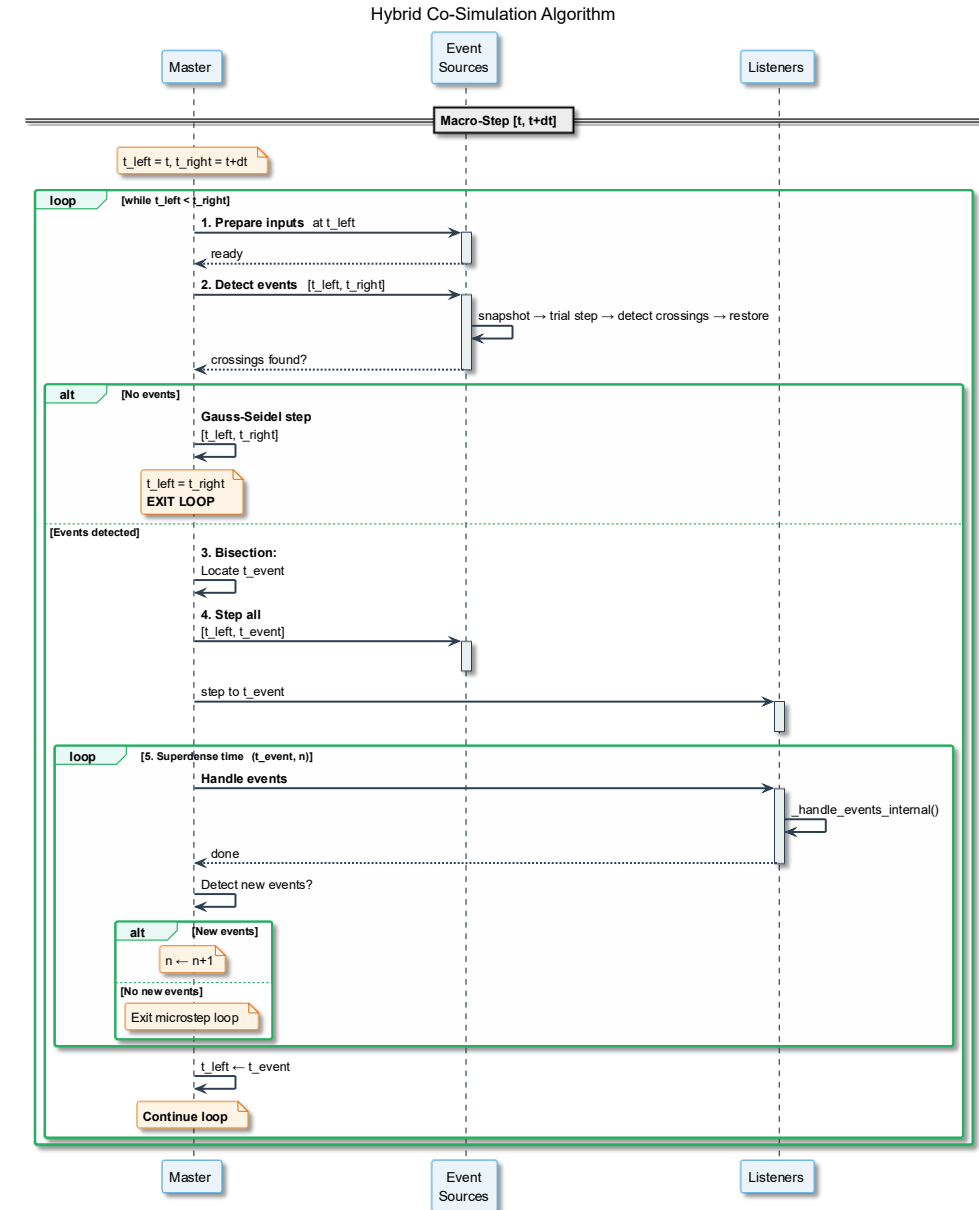
- Each hybrid step processes the interval  $[t, t + dt]$  using a loop that **may split the interval** when events occur
- Inputs of components are set** per execution generation, and **algebraic loops** fully contained in a generation **are solved** with the IJCSA before stepping
- Event detection across event sources:** each event source snapshots its state at  $t_{left}$ , caches its inputs, then executes a trial step to  $t_{right}$  using **`_do_step_internal`** and evaluates event indicators at both ends
- If **no crossings** are found, the algorithm delegates the entire interval to the **Gauss-Seidel algorithm** (enables the use of the most recent input values for later generations)
- If **crossings exist**, the **algorithm locates the earliest event time** within  $[t_{left}, t_{right}]$  via bisection
- All components are stepped to event time  $t_{event}$**  using the **Gauss-Seidel Step**
- All **events** occurring at model time  $t_{event}$  are **resolved with micro step iterations**
- Left interval boundary is updated and continues with the remaining sub-interval

### Event Crossing Detection:

- Take a state snapshot**
- Cache input:** Store current input values
- Evaluate** indicators at  $t_{left}$
- Trial step** to  $t_{right}$
- Evaluate** indicators at  $t_{right}$
- Detect event** crossings (in `component.detect_event_crossings()`)
- Record crossing** to crossings list: (comp\_name, event\_name)
- Rollback:** Restore component to exact state at  $t_{left}$ , restore cached inputs (components are ready for bisection iteration)

### Event Time Localization via Bisection:

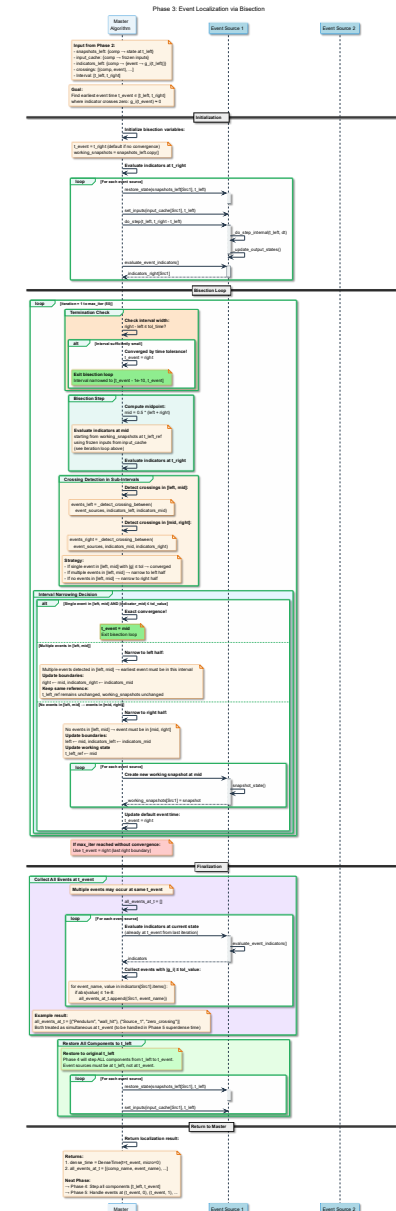
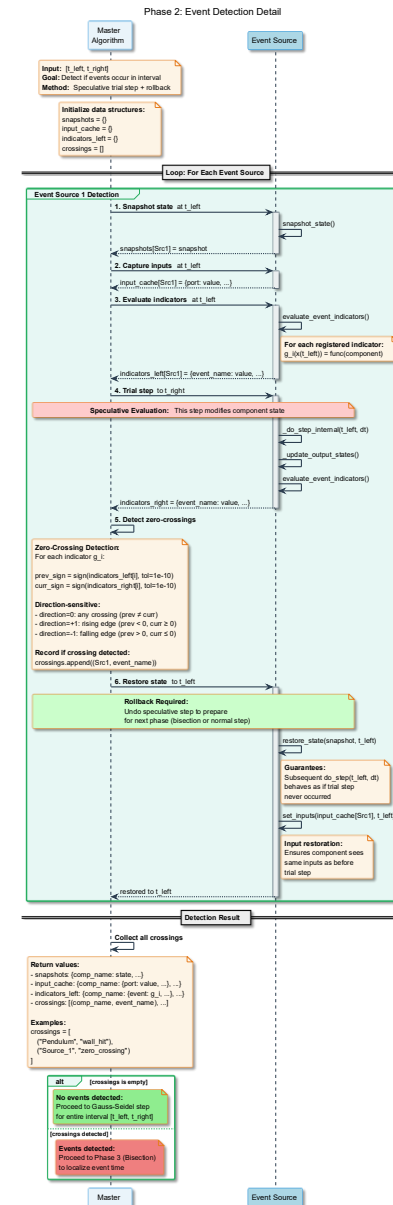
- Check** if the **time step tolerance** is reached
- Compute midpoint** of the interval
- Step all event source** components to the midpoint
- Evaluate indicators** at midpoint
- Detect and record crossings** in the interval  $[left, mid]$  and  $[mid, right]$
- Check for convergence** of the indicator value if there is a **single event in the left interval**
- Narrow left interval** if multiple events in left half
- Narrow right interval** if no events in left half (events must be in interval  $[mid, right]$ ) and update working snapshots



- Event detection caches input at  $t_{left}$  and reuses them during all speculative evaluations (Jacobi step approach)
- Without frozen inputs, each step revision might see different inputs from other components
- Assumption relaxes the requirement that all involved components must support rollback

- Working snapshots of states
  - Are used to keep component state up to date if a crossing occurs in the right half of the interval
  - Next iteration requires to start from the midpoint and not from the original  $t_{left}$
- Termination conditions:
  - **Interval width:**  $t_{right} - t_{left} \leq \epsilon_{time}$
  - **Indicator value:** Finds zero-crossing when indicator is sufficiently close to zero
- If multiple events are found within the defined tolerance, events are collected as strongly simultaneous events

- Handling of events that occur at localized event time (= *current\_time*) in the system
- Current time: instance of **DenseTime** with micro step index 0 is created
- If multiple **events occur with strong simultaneity**, it checks for conflicts:  
Do the event handler touch the same variable and are the event handler commutative?
  - **Event Annotations:** User defines which event handler are commutative (execution order has no impact on final state)
  - **Dynamic Verification:** Executes all permutations of events and checks if the final states are identical. Requires the event listener to support rollback.
- **Dispatch mechanism:**
  - System stores mapping about which component has subscribed which event
  - System calls the CoSimComponent-level event handling method that calls the subclass specific ***\_handle\_events\_internal()*** method
  - Output states and histories are updated
- **Event Chains:** Event at (t, 0) triggers handler that causes new event at (t, 1)
  - Indicators are evaluated before and after event handling to detect crossing due to event handlers, iterate until all chained events are resolved



## Event Detection with Frozen Inputs:

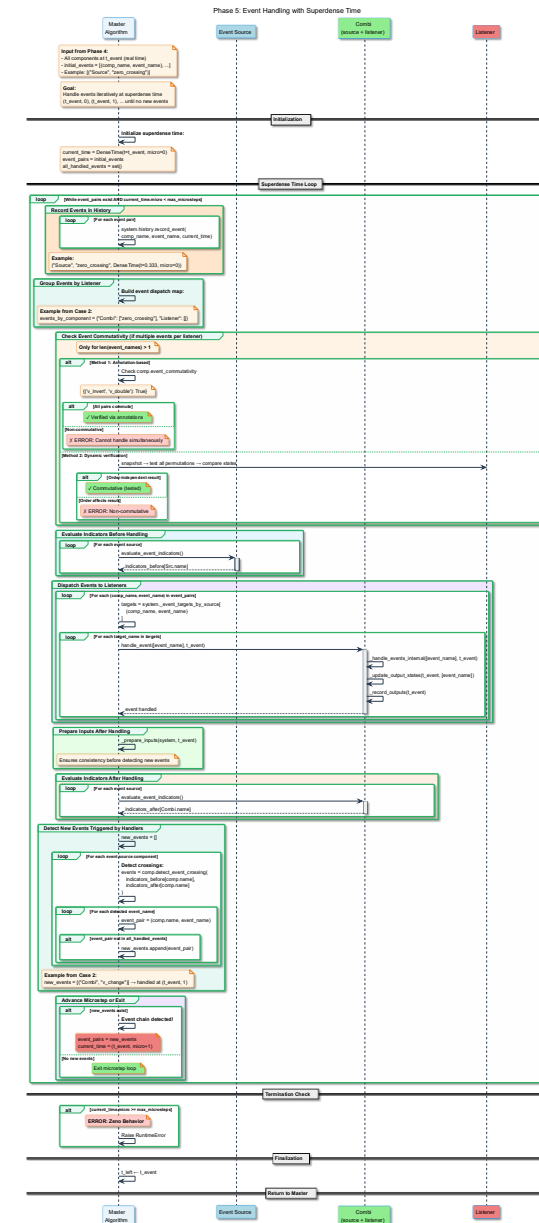
- Event detection caches input at  $t_{left}$  and reuses them during all speculative evaluations (Jacobi step approach)
- Without frozen inputs, each step revision might see different inputs from other components
- Assumption relaxes the requirement that all involved components must support rollback

## Event Localization

- Working snapshots of states
  - Are used to keep component state up to date if a crossing occurs in the right half of the interval
  - Next iteration requires to start from the midpoint and not from the original  $t_{left}$
- Termination conditions:
  - Interval width:**  $t_{right} - t_{left} \leq \epsilon_{time}$
  - Indicator value:** Finds zero-crossing when indicator is sufficiently close to zero
- If multiple events are found within the defined tolerance, events are collected as strongly simultaneous events

## Event Handling:

- Handling of events that occur at localized event time (=  $current\_time$ ) in the system
- Current time: instance of **DenseTime** with micro step index 0 is created
- If multiple **events occur with strong simultaneity**, it checks for conflicts: Do the event handler touch the same variable and are the event handler commutative?
  - Event Annotations:** User defines which event handler are commutative (execution order has no impact on final state)
  - Dynamic Verification:** Executes all permutations of events and checks if the final states are identical. Requires the event listener to support rollback.
- Dispatch mechanism:**
  - System stores mapping about which component has subscribed which event
  - System calls the CoSimComponent-level event handling method that calls the subclass specific **handle\_events\_internal()** method
  - Output states and histories are updated
- Event Chains:** Event at (t, 0) triggers handler that causes new event at (t, 1)
  - Indicators are evaluated before and after event handling to detect crossing due to event handlers, iterate until all chained events are resolved



## FMI 2.0 Capabilities for Hybrid Co-Simulation

- FMU state represents the complete internal state of an FMU during simulation
- State contains:
  - All variable values (continuous, discrete, internal)
  - Solver state
  - Event information
  - Any internal buffers and histories
- API provides functionality to save and restore the state of an FMU
- **getFMUState()**: returns a snapshot of the current FMU state
- **setFMUState(state)**: restores the FMU to a previously captured snapshot
- Enables rollback, checkpointing, and event detection
- **Problem: Implementation of these methods is not mandatory (depends on export tool)**

## Solver-Dependent Behavior

- Available solvers for Co-Simulation FMUs exported from OpenModelica:
  - Explicit Euler: no internal state
  - Ccode: complex multi-step history
- For both solvers, the FMU provides the capability flag **canSetAndGetFMUState=False**
- But **getFMUState / setFMUState()** (FMPy functions) can still be called on the FMU instances
- **Problem:** If the FMU uses the Ccode solver, it cannot re-simulate from a previous time steps as the internal solver state is not aligned with the previous state
- However, if the FMU uses the Explicit Euler solver, it can re-simulate from a previous time step as the internal solver does not maintain a state

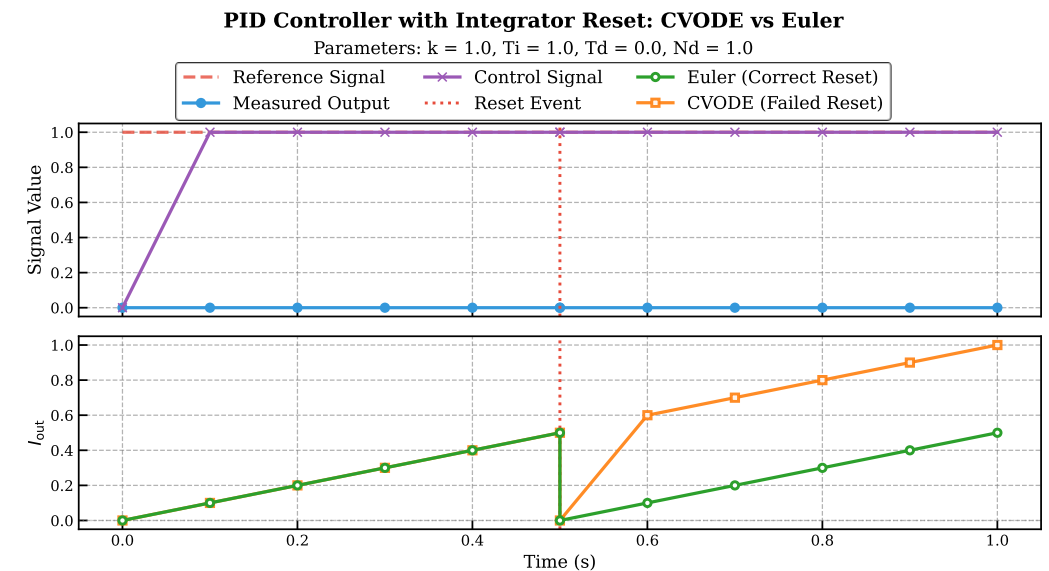
## CVODE:

$$\sum_{i=0}^{K_1} \alpha_{n,i} y^{n-i} + h_n \sum_{i=0}^{K_2} \beta_{n,i} \dot{y}^{n-i} = 0$$

- **Non-stiff (Adams-Moulton):**  $K_1 = 1$ ,  $K_2 = q - 1$ , where  $q$  varies between 1 and 12
- **Stiff (Backward Differentiation Formula):** In the fixed-leading coefficient form (FLC), where  $K_1 = q$ ,  $K_2 = 0$  with  $q$  varying between 1 and 5
- **Multistep method:** Uses history of previous solution approximations ( $y^{n-1}$ ,  $y^{n-2}$ , ...)
- Internal **solver state remains unchanged** after state variable is reset (integration continues as if the reset never occurred)

## Explicit Euler:

- Single step explicit method:  $y^{n+1} = y^n + h \cdot f(t^n, y^n)$
- **Stateless single-step method** that immediately reflects reset changes
- Can be used for systems with discrete zero-time reset changes



(Hindmarsh et al., 2021; Modelica Association, 2024)

## Overview:

- Test case demonstrates the rollback capabilities required for hybrid co-simulation with event localization
- FMI 2.0 standard provides mechanisms for saving and restoring internal FMU states, but availability and behavior depend on the internal solver implementation of the FMU

## FMU Component: Pendulum with Wall Collision

- Discrete event: Wall collision when  $q = 0$  (falling edge)
- Event Handling: Velocity inversion with restitution coefficient:  

$$\omega^+ = -e \cdot \omega^- \text{ with } e = 1$$
- FMU with Euler uses `getFMUState()` / `setFMUState(state)` provided by FMU instance
- FMU with Ccode requires complex workaround
  - Full re-initialization sequence, solver history is cleared
  - It must be possible to define an initial state that corresponds to the state to restore

```
def snapshot_state(self):
    if self.solver == "Euler":
        # returns pointer to internal FMU state
        return self._instance.getFMUState()
    elif self.solver == "Ccode":
        # returns dict with outputs
        return super().get_state()

def restore_state(self, snapshot, t):
    self.t = t
    if self.solver == "Euler":
        self._instance.setFMUState(snapshot)
    elif self.solver == "Ccode":
        # 1) Extract State variable
        q0 = snapshot['q']['value']
        omega0 = snapshot['omega']['value']

        # 2) Full Re-Initialization Sequence
        self._instance.reset()
        self._instance.instantiate()
        self._instance.setupExperiment(startTime=t)
        self._instance.enterInitializationMode()

        # 3) Set initial conditions as parameters
        self.set_parameters(**{'q0': q0, 'omega0': omega0})
        self._apply_parameters_starts()
        self._apply_input_starts()

        # 4) Exit Initialization Mode
        self._instance.exitInitializationMode()
```

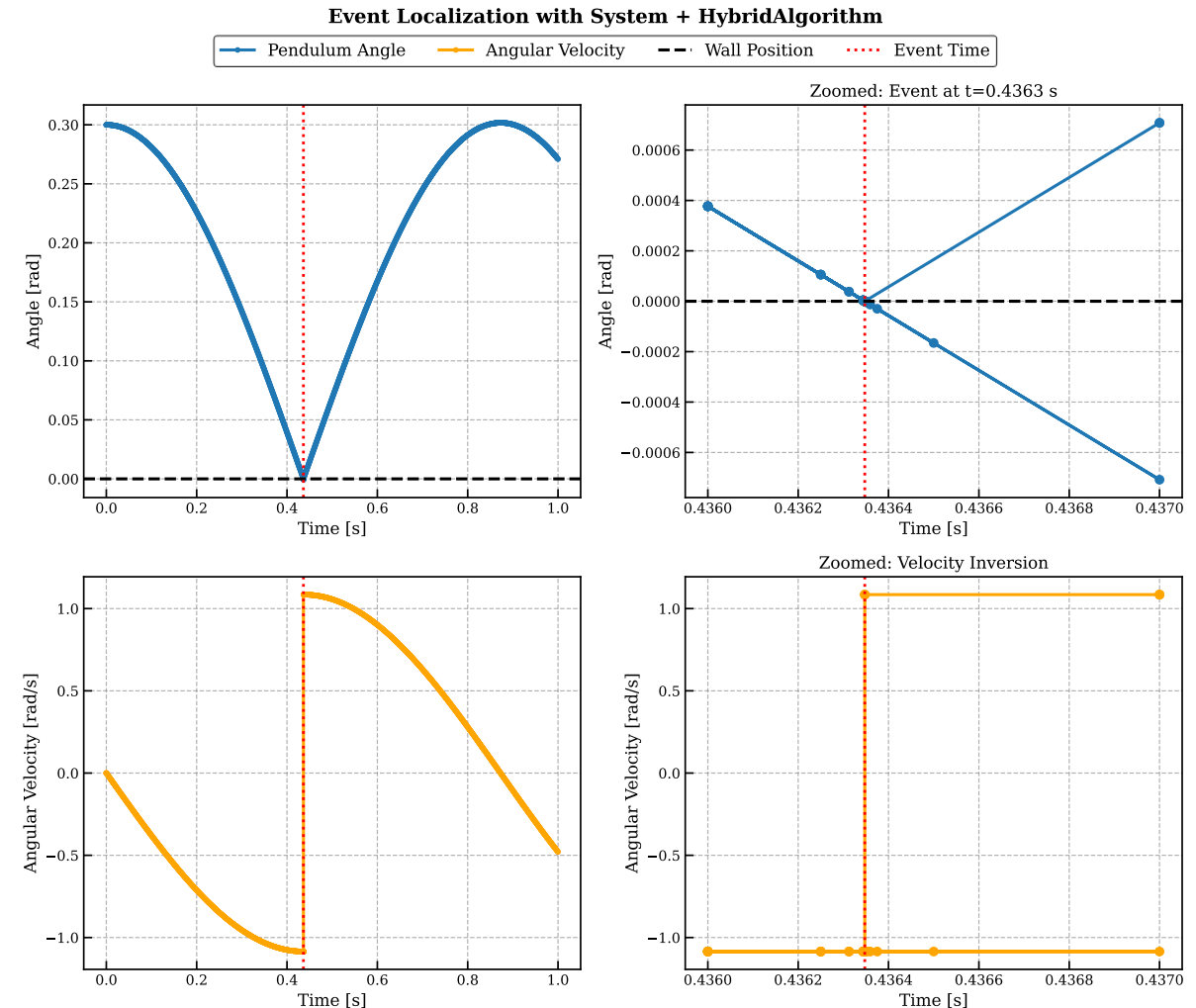
```
# Handle events: invert omega on wall hit
def _handle_events_internal(self, event_names, t):
    if "wall_hit" not in event_names:
        return

    # 1) Get current current state
    output = self.get_state()
    q0 = output['q']['value']
    omega0 = -output['omega']['value']

    # 2) Full Re-Initialization Sequence
    self._instance.reset()
    self._instance.instantiate()
    self._instance.setupExperiment(startTime=t)
    self._instance.enterInitializationMode()

    # 3) Set initial conditions as parameters
    self.set_parameters(**{'q0': q0, 'omega0': omega0})
    self._apply_parameters_starts()
    self._apply_input_starts()

    # 4) Exit Initialization Mode
    self._instance.exitInitializationMode()
```



## System Overview:

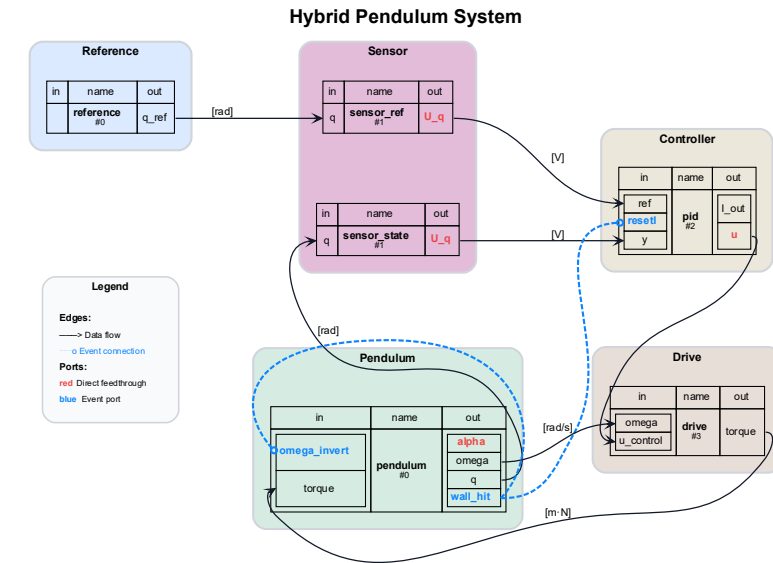
- Test demonstrates a complete hybrid co-simulation of a controlled pendulum system with wall contact detection and event-driven dynamics
- Continuous dynamic behavior of the pendulum between wall contact
- Wall collision creates a discrete event and triggers event handling due to collision
- Event-driven state changes: Angular velocity of the pendulum is inverted and integrator is reset

## Pendulum Class:

- Pendulum class extends FMU component to **support hybrid functionality which is not natively provided by the FMU** exported from OpenModelica
- Event Detection:**
  - Event output port **wall\_hit** is created via **add\_event\_indicator**
  - Event indicator function returns the current angle  $q$  (event detection for falling  $q$ )
- Rollback support for bisection search**
  - Implements **snapshot\_state** and **restore\_state** method for Euler and Ccode solver
- Event Handling:**
  - Event input port **omega\_invert** subscribes to own **wall\_hit** event
  - Handler logic: velocity inversion with restitution
  - Reinitialization is needed since velocity is a state variable which cannot be directly modified
  - Initial condition ( $q_0, \omega_0$ ) are parameters not state variables
  - Workaround: Treat event as new simulation start with modified parameters
- Event Port Management: Uses Boolean signals and is True during event occurrence and false otherwise (active only at event time instants)

## PID Class

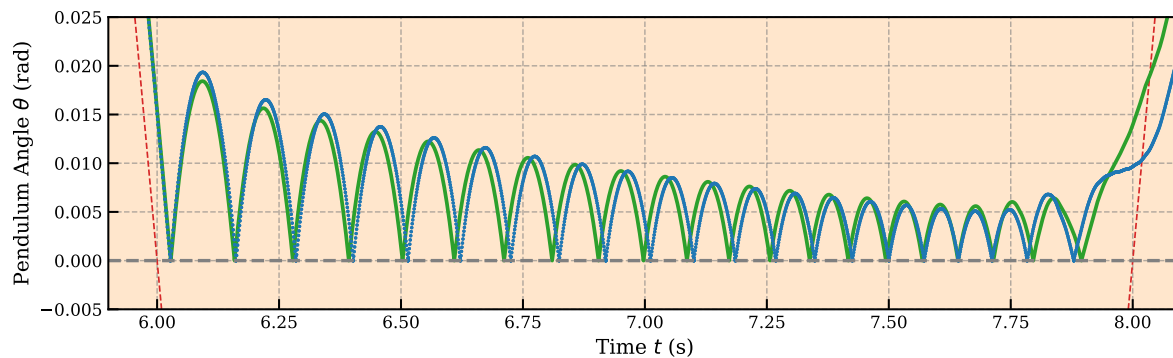
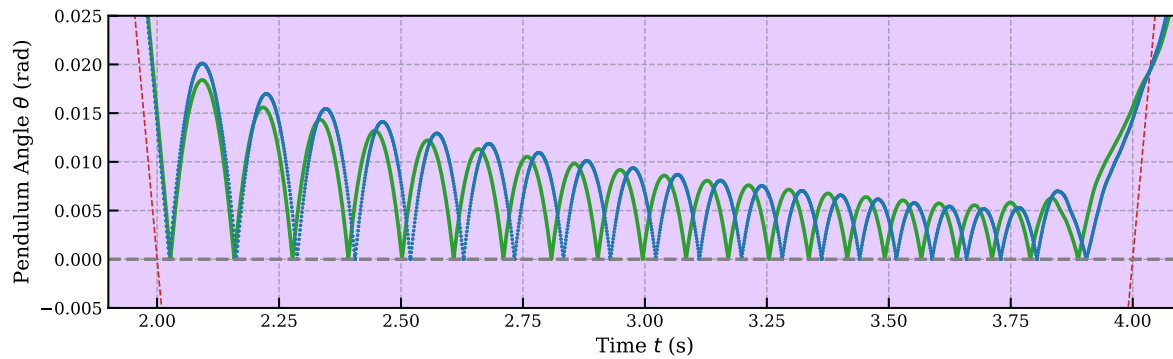
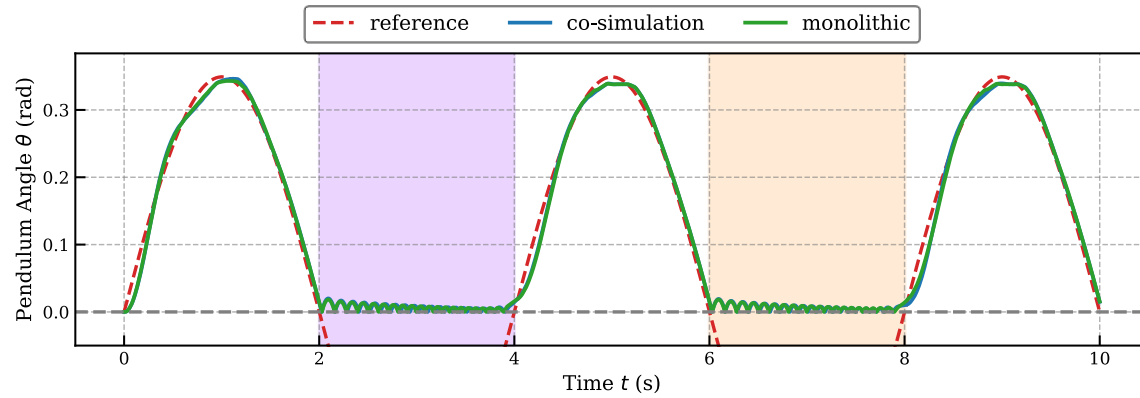
- Extends the FMUComponent to support the required integrator reset capabilities
- Event input port **reset\_I** (FMU parameter for integrator reset)
- Handler logic: setting the input flag for integrator reset **reset\_I** true, performs a zero-length step to apply the reset, and clear reset signal afterwards
- No rollback needed since the PID is listener only, not event source



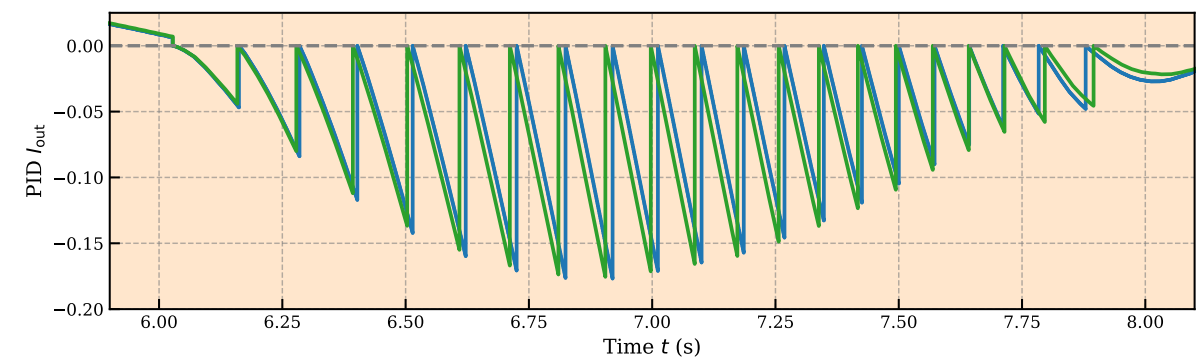
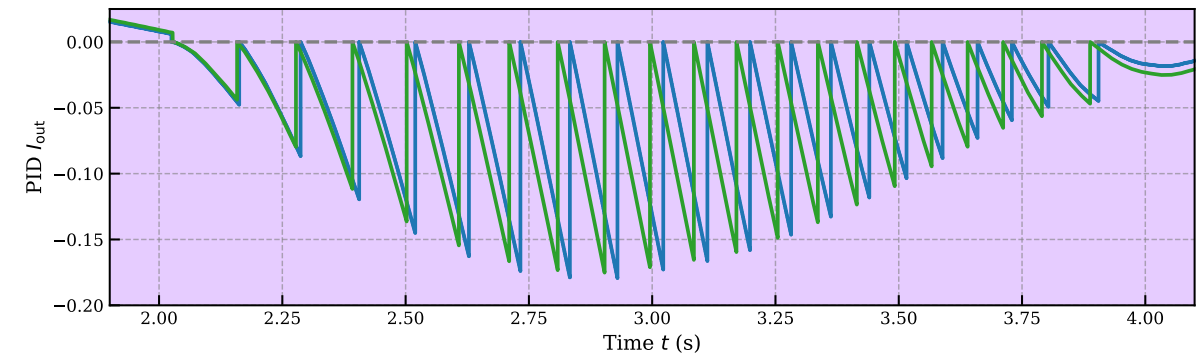
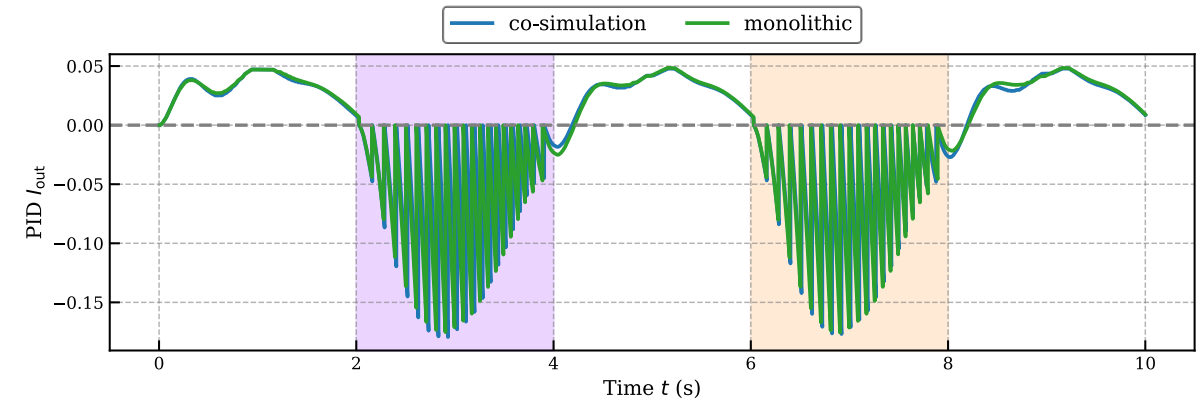
```

1 class PID(FMUComponent):
2     """
3     FMU-based PID controller component with integrator reset on events.
4     """
5     def __init__(self, name, group = "Controller"):
6         fmu_path = Path.cwd().parent / "FMUs" / f"PID_Continuous_Euler.fmu"
7         super().__init__(name, fmu_path, group)
8
9     def _handle_events_internal(self, event_names, t):
10        if "wall_hit" not in event_names:
11            return
12        self.set_inputs({'resetI': True})
13        self.do_step(t, 0) # Perform a step to apply the resetI input
14        self.set_inputs({'resetI': False})
    
```

Pendulum Angle: Monolithic vs. Co-Simulation



PID Integrator Output: Monolithic vs. Co-Simulation



## Core Capabilities

### 1. Event Detection and Zero-Crossing Mechanism

- Components register callable functions that return scalar values; zero-crossings trigger events
- Components can register multiple event indicators simultaneously

### 2. Event Time Localization via Bisection

- Framework uses **iterative bisection** to precisely locate the event time
- Locates earliest event when multiple crossings detected

### 3. Superdense Time and Event Chains

- Enables handling of **event chains** where one event triggers another at the same real time
- Events at micro step  $n$  can only affect components at micro step  $n+1$  or later.

### 4. Simultaneous Event Conflict Resolution

- When multiple events occur at the **same real time** (within tolerance), the framework ensures **deterministic execution** via commutativity checking
- Annotation-based verification and dynamic permutation testing

### 5. State Rollback and Restoration

- Framework requires **event source components** (those with event indicators) to implement rollback (during event detection, bisection iterations)
- Components that only *react* to events (listeners) do not require rollback capabilities

### 6. Event Dispatching and Subscription Model

- Framework uses an **explicit subscription model** for event handling
- **System dispatches events** based on internal stored mapping and calls *handle\_event()* on each target component

### 7. Component Classification

- **Event Sources:** has state events, and must support rollback
- **Event Listeners:** React on state events, provide event handling logic
- **Continuous Only:** Do not provide any hybrid capabilities

### 8. Hybrid Co-Simulation Algorithm

- **Auto-Selection:** If system contains event sources, hybrid algorithm is automatically selected.
- Extends the Gauss-Seidel Algorithm with Event Handling

## Current Limitations

### 1. Floating-Point Time and Transitivity of Simultaneity

- Using floating-point time with tolerance-based simultaneity detection violates transitivity:
  - If events A and B are simultaneous (within tolerance)
  - And events B and C are simultaneous (within tolerance)
  - Then A and C may **not** be simultaneous
- Three or more events with overlapping tolerance intervals may be incorrectly classified as simultaneous.
- **Proposal:** Integer-based time representation with resolution negotiation (Cremona et. al)

### 2. Time Events as State Events

- Strict time-based events (e.g., periodic sampling) must be encoded as state events
- Less efficient than native time-event support; requires knowledge of event schedule.
- **Proposal:** Add explicit *add\_time\_event()* method (without bisection for localization)

### 3. True-Gauss Seidel Incompatibility for Event Listeners

- True Gauss-Seidel hybrid co-simulation cannot be implemented because:
  - Earlier-generation event listeners would need to rollback to event time
  - Event listeners are not required to support rollback
  - Framework adopts **Jacobi-like semantics**: all event sources step to event time together (no exchange of data)
- **Trade-off:** Flexibility (listeners without rollback) vs. performance (generation pipelining).

### 4. Hybrid Master Pendulum

- **Status:** Not yet adapted for hybrid co-simulation.
- **FEM Pendulum:** already updated
- **Required Updates:**
  - **OpenSim Pendulum:** implement rollback capability
  - **Master Pendulum:** Hybrid functionality on multi-component level
- **Goal:** Validate hybrid heterogeneous controlled pendulum system with *MasterPendulum*

### 5. Max Micro Step Safeguard

- If maximum micro steps are reached **RuntimeError** is risen (no distinction if invalid)
- **Proposal:** Add cycle detection to identify true cyclic event dependencies

## Framework Core Expansion:

- **New Components:**
  - OpenSimComponent: General Component Class for OpenSim models ✓
  - FEMComponent: General Component Class for netgen/NGSolve FEM models
- **Multicomponent Concept:**
  - Combine several representations of the same “thing” (e.g., pendulum FMU + OpenSim + FEM)
  - Implement state synchronization and rule-based switching between models ✓
- **Hybrid Capabilities:** ✓
  - Support discrete and continuous states
  - Introduce hybrid DAE event handling (zero-crossing detection, event-triggered switching)
- **File-based (FEM) simulation tool integration concept** ⌚
  - Develop a workflow for file-based (FEM) model integration (openCFS, CalculiX, etc.)

## Simulation Engine & Computational Causality:

- **Goal: Enable robust, adaptive, and event-driven co-simulation**
- **Multi-rate / Adaptive Time Stepping:** ✓
  - Each component advances with its local solver time base
  - synchronized at communication points
- **Causality Solver:** ✓
  - Extend execution order computation to handle cyclic dependencies
  - Algebraic loop resolution

## User Interaction & System Assembly

- **Goal: Provide an intuitive, guided workflow inside Jupyter**
- **Multi-Step Wizard Interface (Standard & Expert Mode)**
  - Model selection -> FMU Export -> system assembly -> configuration -> simulation
  - Live preview of Modelica code ⌚
- **Configuration Management:**
  - Save and reload full system configurations (components, connections, parameters)
  - Implement configuration validation
- **Visualization:** ✓
  - Generate system graphs
  - Enable live plotting and simulation diagnostics for monitoring
- **Result Handling:** ⌚
  - Standardize simulation metadata and result exports for traceability

## Framework Integration & Validation

- **Goal: Consolidate core modules and ensure framework stability** ✓
- **Testing and Automation:**
  - Update and expand unit tests for all framework layers
  - Unit tests for core classes, wrappers, and port connections
  - Integration tests for system assembly and simulated run
  - Cross-platform validation (Windows/Linux/Mac)
- **Data Layer:** Structured result logging for reproducible simulation data ⌚
- **Documentation and Packaging:**
  - Refactor into modular pyproject-based package
  - Add tutorials, notebook, and licensing for distribution

1. A. C. Hindmarsh, R. Serban, C. J. Balos, D. J. Gardner, D. R. Reynolds, and C. S. Woodward, **User Documentation for CVODE (SUNDIALS v5.7.0)**, Lawrence Livermore National Laboratory, Feb. 2021. [Online]. Available: [https://sundials.readthedocs.io/en/latest/cvode/Mathematics\\_link.html](https://sundials.readthedocs.io/en/latest/cvode/Mathematics_link.html)
2. Modelica Association, **Functional Mock-up Interface for Model Exchange and Co-Simulation**, Version 2.0.5, Nov. 2024. [Online]. Available: <https://fmi-standard.org/>
3. F. Cremona, M. Lohstroh, D. Broman, M. Di Natale, E. A. Lee and S. Tripakis, **"Step revision in hybrid Co-simulation with FMI"**, 2016 ACM/IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE), Kanpur, India, 2016, pp. 173-183, doi: 10.1109/MEMCOD.2016.7797762.
4. Cremona, Fabio & Lohstroh, Marten & Broman, David & Lee, Edward & Masin, Michael & Tripakis, Stavros. (2018). **Hybrid Co-simulation: It's About Time**. MODELS '18: Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems. 368-368. 10.1145/3239372.3242896.
5. Urquía Moraleda, A., & Martín Villalba, C. (2018, April). **Modeling and simulation in engineering using Modelica**. Editorial UNED. <https://espacio.uned.es/entities/publication/cbfb24a9-2ec0-4afa-aa9c-243e1309da6e/full>
6. Broman, D., Greenberg, L., Lee, E.A., Masin, M., Tripakis, S., Wetter, M.: **Requirements for hybrid cosimulation standards**. In: Proceedings of 18th ACM International Conference on Hybrid Systems: Computation and Control (HSCC), pp. 179–188. ACM (2015)